

Higham Exercise Benchmark Candidates

LeanFpAnalysis Benchmarking

June 24, 2026

1 Purpose

This document collects candidate exercises for the LeanFpAnalysis benchmark. It is a benchmark ledger, not a source-coverage ledger.

The benchmark goal is to test whether access to the LeanFpAnalysis library makes formalizing floating-point stability-analysis tasks faster and more reliable than working from a minimal statement scaffold. Preferred exercises ask for perturbation bounds, backward-error bounds, forward-error bounds, residual bounds, condition numbers, or algorithmic stability statements. We avoid tasks that are only lookup, empirical exploration, or exact algebra unless they directly support a stability bound.

Long verbatim exercise statements are intentionally not copied here. Each entry records the exercise number, a source-faithful prompt paraphrase with the essential equations or bound target, and placeholders for the Lean task statement and generated proofs.

The early-chapter numbering follows the user-curated exercise list. In a few places the local PDF extraction differs slightly from the screenshot numbering; the prompt text identifies the intended exercise content.

2 Benchmark Design

Condition A Minimal statement scaffold. This includes only what is needed to state the exercise in Lean in the same form as Condition C: basic types, notation, local structures, and possibly concrete algorithm definitions. It must not include stability bounds, source-derived proof lemmas, or proof logic.

Condition C The same task file and the same neutral statement scaffold, plus the LeanFpAnalysis library context and documentation.

Each exercise is evaluated in two layers.

Layer 1: natural-language discovery Both conditions receive the exercise prompt and, if needed, the same neutral statement scaffold. Condition A has no stability-library help, while Condition C sees the LeanFpAnalysis library and documentation. This tests whether library context helps the model choose a good Lean formalization and prove it. The result is inherently noisy because the two runs may choose different theorem statements.

Layer 2: fixed Lean theorem A canonical Lean theorem statement is prepared and frozen before benchmarking. Both conditions receive exactly this theorem. Condition A receives the minimal statement scaffold only; Condition C receives the same theorem and scaffold plus LeanFpAnalysis context. This is the primary benchmark because it tests proof support without theorem-selection drift.

For both layers, paste the generated proof under the corresponding condition. For Layer 1, also record whether the generated Lean statements are genuinely the same mathematical claim. For Layer 2, the theorem statement must be byte-for-byte identical across conditions.

3 Code Block Convention

Use Lean listings for task statements and proofs:

```
theorem example_task : True := by
  trivial
```

4 Current Library Coverage Notes

- Rounded `exp`, rounded `log`, and `log1p` models are not yet a settled part of the library. Exercises that need these operations should remain in the benchmark: they can show whether the model can introduce adequate local stubs, whether a proof exposes a real library extension need, or whether the current library context is insufficient. If a run scopes such a task to exact algebra, that scoping must be recorded explicitly in the assessment rather than treated as the original task.

5 Candidate Index

Chapter	Exercise	Tier	Theme	Benchmark intent
1	1.9	Best	Forward and backward error	Cramer’s rule for 2 by 2 systems; derive forward and residual bounds.
1	1.10	Good	Forward error	Two-pass sample variance error bound.
1	1.7	Good	Condition number	Componentwise and normwise condition numbers for sample variance.
1	1.5	Maybe	Accurate reformulation	Large- n computation of $(1 + 1/n)^n$; useful probe for rounded log/exp support.
3	3.5	Best	Backward error	Matrix multiplication backward error with perturbation in either factor.
3	3.2	Good	Gamma bound	Sharpened product-error gamma bound under special hypotheses.
3	3.8	Good	Matrix product forward error	Bound for product of a sequence of matrices.
3	3.12	Good	Quadrature error	Floating-point summation and function evaluation error bound for quadrature.
3	3.7	Maybe	Complex backward error	Complex analogues of real backward-error results.
4	4.3	Best	Summation forward error	Refined recursive-summation error representation and ordering bound.

Chapter	Exercise	Tier	Theme	Benchmark intent
4	4.7	Maybe	Stability comparison	Analyze summation via $\log(\prod_i \exp x_i)$; useful probe for rounded exp/log support.
4	4.8	Maybe	Grid generation error	Compare recurrence and direct formulas for equally spaced grid points.
5	5.2	Best	Polynomial evaluation error	Error analysis of beginner polynomial evaluation.
5	5.3	Best	Horner-style error bound	Error bound for even/odd splitting with Horner on each part.
5	5.6	Best	Polynomial backward/forward error	Bound for computed polynomial from Horner-like evaluation.
5	5.5	Good	Product-form evaluation	Error analysis for root-product evaluation of a polynomial.
7	7.2	Good	Residual-to-forward error	Relate residual, forward error, and condition number for linear systems.
7	7.7	Good	Backward error comparison	Compare backward-error quantities under perturbed right-hand sides.
8	8.3	Best	Triangular solve forward error	Bound components of triangular solve under unit upper triangular assumptions.
8	8.4	Best	Componentwise conditioning	Bound triangular M-matrix solve condition measure.
8	8.7	Maybe	Diagonal dominance bound	Use diagonal dominance to bound inverse norms; use scoped part only.
9	9.6	Good	Growth factor	Totally nonnegative matrices and growth factor one.
9	9.8	Good	LU backward structure	Inverse totally nonnegative case and absolute LU factorization.
9	9.9	Good	Growth factor bound	Bound GE growth factor without pivoting using $ L U $.
9	9.13	Good	Threshold pivoting	Growth-factor bound for sparse threshold pivoting.
9	9.14	Maybe	Pivoting equivalence	Pre-pivoted GEPP equivalences; mostly structural.
19	19.2	Best	Orthogonality error	Bound $\widehat{P}^T \widehat{P} - I$ for a computed Householder reflector.
19	19.6	Best	Householder update error	Derive one-stage Householder QR update error and row-scaling interpretation.
19	19.10	Good	Gram-Schmidt instability	Analyze orthogonality of CGS and MGS on a small Lauchli-type matrix.
19	19.12	Good	Backward-error structure	Convert block perturbation data into a QR backward-error statement.
20	20.2	Best	Least-squares backward stability	Prove or scope-formalize Householder QR LS backward-stability theorem.
20	20.5	Best	MGS LS backward stability	Analogue of the QR LS backward-stability result for MGS.
20	20.6	Best	Residual error	Bound difference between true and computed LS residual norms for normal equations.

Chapter	Exercise	Tier	Theme	Benchmark intent
20	20.8	Good	LS backward error	Special case of LS backward-error formula when the right-hand side is unperturbed.
20	20.9	Good	Stable formula	Derive cancellation-resistant formula for normwise LS backward error.
20	20.11	Good	First-order approximation	Show LSE condition expression reduces to unconstrained LS first-order form.
22	22.8	Best	Bidiagonal inverse perturbation	Derive exact inverse perturbation formula and componentwise bound.
22	22.11	Best	Structured condition number	Derive structured condition number for primal Vandermonde systems.
22	22.3	Good	Algorithm stability	Scoped residual or forward-error task for Vandermonde inverse algorithm.
22	22.7	Good	Conditioning	Condition bounds for Chebyshev-Vandermonde matrices at special nodes.
23	23.6	Best	Fast matrix multiply error	Prove 3M plus Strassen normwise error bound.
24	24.1	Best	FFT convolution error	Bound DFT convolution rounding error and integer recovery condition.
25	25.1	Good	Perturbed iteration stability	Error floor for contraction iteration with bounded perturbations.
27	27.5	Good	Robust norm computation	Correctness and stability properties of scaled 2-norm algorithm.
27	27.6	Good	Robust Pythagorean sum	Halley/pythag iteration, convergence, and robustness properties.

6 Exercise Workspaces

6.1 Ch01 Ex. 1.9 – Cramer’s Rule Stability

Tier Best

Theme Forward and residual error

Exercise prompt For a 2 by 2 system $Ax = b$, Cramer’s rule forms

$$d = a_{11}a_{22} - a_{21}a_{12}, \quad x_1 = (b_1a_{22} - b_2a_{12})/d, \quad x_2 = (a_{11}b_2 - a_{21}b_1)/d.$$

Assuming d is exact, show that the computed $\hat{x}$ satisfies

$$\frac{\|x - \hat{x}\|_\infty}{\|x\|_\infty} \leq \gamma_3 \operatorname{cond}(A, x), \quad \|b - A\hat{x}\|_\infty \leq \gamma_3 \operatorname{cond}(A^{-1})\|b\|_\infty,$$

with the componentwise condition quantities defined using $|A^{-1}||A||x|$.

Benchmark intent Derive forward and residual bounds from the stated operation model.

Expected dependencies

Floating-point product/sum error; small linear-system perturbation bounds.

Notes Good first benchmark because the task is small but not a lookup.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.2 Ch01 Ex. 1.10 – Two-Pass Sample Variance

Tier Good

Theme Forward error

Exercise prompt For the two-pass sample variance formula $V(x)$, prove that the computed quantity $\widehat{V} = \text{fl}(V(x))$ obeys

$$\frac{|V - \widehat{V}|}{V} \leq (n + 3)u + O(u^2).$$

The point is that the leading term should not depend on the sample-variance condition numbers from Ex. 1.7.

Benchmark intent Show the relative error is bounded by a linear term in nu plus higher-order terms.

Expected dependencies

Summation error; dot/product error; variance definitions.

Notes More valuable after sample-variance and summation APIs are settled.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.3 Ch01 Ex. 1.7 – Sample Variance Condition Numbers

Tier Good

Theme Condition number

Exercise prompt For sample variance $V(x)$, define a componentwise condition number using $|\Delta x_i| \leq \epsilon |x_i|$ and show

$$\kappa_C = \frac{2 \sum_i |x_i - \bar{x}| |x_i|}{(n-1)V(x)}.$$

For the normwise perturbation set $\|\Delta x\|_2 \leq \epsilon \|x\|_2$, show

$$\kappa_N = \frac{2\|x\|_2}{\sqrt{(n-1)V(x)}} = 2 \left(1 + \frac{n}{n-1} \frac{\bar{x}^2}{V(x)} \right)^{1/2} \geq \kappa_C.$$

Benchmark intent Formalize the condition-number formulas and comparison.

Expected dependencies

Exact calculus or first-order perturbation framework; variance definitions.

Notes Useful companion to Ch01 Ex. 1.10.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.4 Ch01 Ex. 1.5 – Accurate Large-n Power

Tier	Maybe
Theme	Accurate reformulation
Exercise prompt	First obtain an accurate method for $\log(1+x)$ for all $x > -1$, including small $ x $, assuming the library <code>log</code> has relative error at most u . Then use $(1 + 1/n)^n = \exp(n \log(1 + 1/n))$ to compute $(1 + 1/n)^n$ accurately for large n .
Benchmark intent	Test whether the run introduces adequate rounded <code>log/exp</code> stubs or exposes a library extension need.
Expected dependencies	Rounded <code>log/exp</code> models; possible local <code>log1p</code> specification.
Notes	Keep in the benchmark as a coverage probe. If a run proves only an exact algebra variant, record that explicitly as a scoped result.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No `LeanFpAnalysis` imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with `LeanFpAnalysis` imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining sorrys, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.5 Ch03 Ex. 3.5 – Matrix Multiplication Backward Error

Tier Best

Theme Backward error

Exercise prompt For nonsingular $A, B \in \mathbb{R}^{n \times n}$, show that the computed product can be written as

$$\mathbf{fl}(AB) = (A + \Delta A)B, \quad |\Delta A| \leq \gamma_n |A| |B| |B^{-1}|.$$

Also derive the corresponding backward-error representation in which B is perturbed instead of A .

Benchmark intent Prove a componentwise or normwise backward-error statement for matrix multiplication.

Expected dependencies

Dot-product backward error; matrix-vector or matrix-matrix composition.

Notes Excellent core benchmark.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

-- TODO: paste model output from natural-language Condition A run

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

-- TODO: paste model output from natural-language Condition C run

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions

Minimal statement context

-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof lemmas

Condition A proof

-- TODO: generated proof from fixed Lean theorem with minimal statement scaffold

Condition C library context

-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C

Condition C proof

-- TODO: generated proof from the exact same fixed Lean theorem with library context

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.6 Ch03 Ex. 3.2 – Sharpened Product Gamma Bound

Tier Good

Theme Gamma bound

Exercise prompt In Lemma 3.1, specialize to the case $\rho_i \equiv 1$. Prove the sharper product-error estimate

$$|\theta_n| \leq \frac{nu}{1 - \frac{1}{2}nu}, \quad nu < 2.$$

Benchmark intent Prove the strengthened gamma inequality.

Expected dependencies

Gamma algebra and product-of-errors lemmas.

Notes Calibration task for gamma infrastructure.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.7 Ch03 Ex. 3.8 – Matrix Sequence Product Error

Tier Good

Theme Matrix product forward error

Exercise prompt For $A_1, \dots, A_k \in \mathbb{R}^{n \times n}$, bound the forward error in the computed matrix product by proving a statement of the form

$$\|A_1 \cdots A_k - \text{fl}(A_1 \cdots A_k)\|_F \leq (kn^2u + O(u^2))\|A_1\|_2 \cdots \|A_k\|_2.$$

Benchmark intent Show a Frobenius or normwise error bound for a finite matrix product.

Expected dependencies

Matrix multiplication error and norm composition.

Notes Good if scoped to fixed product length or a clean recursive product definition.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No `LeanFpAnalysis` imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.8 Ch03 Ex. 3.12 – Quadrature Rule Error

Tier	Good
Theme	Quadrature error
Exercise prompt	For a quadrature rule

$$I(f) = \int_a^b f(x) dx \approx J(f) = \sum_{i=1}^n w_i f(x_i),$$

assume w_i, x_i are floating-point data, the weighted sum is evaluated left-to-right, and $\text{fl}(f(x_i)) = f(x_i)(1 + \eta_i)$, $|\eta_i| \leq \eta$. Derive and interpret a bound for $|I(f) - \widehat{J}(f)|$, where $\widehat{J}(f) = \text{fl}(J(f))$.

Benchmark intent Derive and interpret a bound for $|I(f) - \widehat{J}(f)|$.

Expected dependencies

Summation bounds; weighted sum bounds.

Notes Good applied stability task.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.9 Ch03 Ex. 3.7 – Complex Backward Error Analogues

Tier	Maybe
Theme	Complex backward error
Exercise prompt	Give complex analogues of the real backward-error results for inner products and matrix-vector products, replacing transposes by conjugate transposes where appropriate and using complex modulus in the componentwise bounds.
Benchmark intent	Prove one specific complex dot-product or matrix-vector backward-error analogue.
Expected dependencies	Complex absolute values; real/complex transfer lemmas.
Notes	Avoid using it as a broad prose task.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No `LeanFpAnalysis` imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

-- TODO: paste model output from natural-language Condition A run

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

-- TODO: paste model output from natural-language Condition C run

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions

Minimal statement context

-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof lemmas

Condition A proof

-- TODO: generated proof from fixed Lean theorem with minimal statement scaffold

Condition C library context

-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C

Condition C proof

-- TODO: generated proof from the exact same fixed Lean theorem with library context

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.10 Ch04 Ex. 4.3 – Recursive Summation Refined Bound

Tier Best

Theme Summation forward error

Exercise prompt Let $S_n = \sum_{i=1}^n x_i$ be computed by natural-order recursive summation. Show the refined representation

$$\hat{S}_n = (x_1 + x_2)(1 + \theta_{n-1}) + \sum_{i=3}^n x_i(1 + \theta_{n-i+1}), \quad |\theta_k| \leq \gamma_k = \frac{ku}{1 - ku}.$$

Deduce

$$|\hat{S}_n - S_n| \leq (|x_1| + |x_2|)\gamma_{n-1} + \sum_{i=3}^n |x_i|\gamma_{n-i+1},$$

and determine the ordering of the x_i that minimizes this bound.

Benchmark intent Prove the displayed weighted gamma bound and ordering consequence.

Expected dependencies

Recursive summation backward error and gamma monotonicity.

Notes Strong because the proof is stability-specific and not just algebra.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.11 Ch04 Ex. 4.7 – Log-Exp Summation

Tier Maybe

Theme Stability comparison

Exercise prompt Analyze the accuracy of computing a sum through the identity

$$S_n = \log \left(\prod_{i=1}^n e^{x_i} \right).$$

The intended task is to compare this with direct summation and expose the rounding-error risks introduced by rounded exp, product accumulation, and rounded log.

Benchmark intent Test whether the run can model rounded exp/log plus product accumulation, or identify the missing operation model cleanly.

Expected dependencies

Rounded exp/log models; product error.

Notes

Keep in the benchmark as a coverage probe. If a run proves only an exact algebra variant, record that explicitly as a scoped result.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.12 Ch04 Ex. 4.8 – Grid Point Formation

Tier Maybe

Theme Grid generation error

Exercise prompt For equally spaced points on $[a, b]$,

$$x_i = a + ih, \quad h = (b - a)/n, \quad i = 0, \dots, n,$$

compare the accuracy of forming x_i by repeated addition $x_i = x_{i-1} + h$, by the direct formula $a + ih$, and by the convex combination $a(1 - i/n) + (i/n)b$. Assume a, b are floating-point numbers but h need not be.

Benchmark intent Prove one scoped error comparison.

Expected dependencies

Rounded addition/multiplication and simple recurrences.

Notes Keep the task narrow.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.13 Ch05 Ex. 5.2 – Beginner Polynomial Evaluation

Tier Best

Theme Polynomial evaluation error

Exercise prompt For $p(x) = a_0 + a_1x + \dots + a_nx^n$, analyze the direct evaluator

$$q := a_0, \quad y := 1, \quad \text{for } i = 1 : n \text{ do } y := xy, \quad q := q + a_iy.$$

Derive a rounding-error bound for the returned q .

Benchmark intent Prove a coefficientwise or normwise error bound.

Expected dependencies

Product recurrence; summation error; polynomial definitions.

Notes Good contrast with Horner tasks.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No `LeanFpAnalysis` imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

-- TODO: paste model output from natural-language Condition A run

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

-- TODO: paste model output from natural-language Condition C run

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions

Minimal statement context

-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof lemmas

Condition A proof

-- TODO: generated proof from fixed Lean theorem with minimal statement scaffold

Condition C library context

-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C

Condition C proof

-- TODO: generated proof from the exact same fixed Lean theorem with library context

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.14 Ch05 Ex. 5.3 – Even/Odd Polynomial Splitting

Tier	Best
Theme	Horner-style error bound
Exercise prompt	Let $p(x) = a_0 + a_1x + \cdots + a_nx^n$ with $n = 2m$. Write $p(x) = a_0 + a_2y + \cdots + a_{2m}y^m + x(a_1 + a_3y + \cdots + a_{2m-1}y^{m-1}), \quad y = x^2.$ Obtain an error bound for $\text{fl}(p(x))$ when each part is evaluated by Horner's rule and the final combination is rounded.
Benchmark intent	Derive an error bound for the split evaluation.
Expected dependencies	Horner backward error; product error; polynomial splitting.
Notes	Strong because it composes existing stability lemmas.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context


```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.15 Ch05 Ex. 5.6 – Computed Polynomial Bound

Tier Best

Theme Polynomial backward/forward error

Exercise prompt For the polynomial P_3 in the surrounding chapter, prove that the computed polynomial $\hat{P}_3$ produced by Horner’s method satisfies

$$\|P_3 - \hat{P}_3\| \leq n(m+1)u \tilde{p}_3(\|X\|) + O(u^2), \quad \tilde{p}_3(x) = \sum_k \|A_k\| x^k,$$

for the 1-norm or ∞ -norm.

Benchmark intent Formalize the norm bound involving a nonnegative majorant polynomial.

Expected dependencies

Horner coefficientwise backward error and norm bounds.

Notes Strong higher-level Horner benchmark.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No `LeanFpAnalysis` imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

-- TODO: paste model output from natural-language Condition A run

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

-- TODO: paste model output from natural-language Condition C run

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions

Minimal statement context

-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof lemmas

Condition A proof

-- TODO: generated proof from fixed Lean theorem with minimal statement scaffold

Condition C library context

-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C

Condition C proof

-- TODO: generated proof from the exact same fixed Lean theorem with library context

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.16 Ch05 Ex. 5.5 – Root Product Evaluation

Tier Good

Theme Product-form evaluation

Exercise prompt If $p(x) = \sum_{i=0}^n a_i x^i$ has roots $x_1, \dots, x_n$, evaluate it by the product form

$$p(x) = a_n \prod_{i=1}^n (x - x_i).$$

Give a rounding-error analysis for this product evaluation.

Benchmark intent Derive a product-form backward or forward error bound.

Expected dependencies

Product rounding lemmas.

Notes Good if roots and polynomial product form are already available.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.17 Ch07 Ex. 7.2 – Residual and Forward Error

Tier Good

Theme Residual-to-forward error

Exercise prompt Let $Ax = b$, $A \in \mathbb{R}^{n \times n}$, and let y be any approximate solution with residual $r = b - Ay$. For any subordinate matrix norm, prove

$$\frac{\|r\|}{\|A\|\|x\|} \leq \frac{\|x - y\|}{\|x\|} \leq \kappa(A) \frac{\|r\|}{\|A\|\|x\|}.$$

Interpret the bound.

Benchmark intent Prove the two-sided residual/forward-error inequality.

Expected dependencies

Normed matrix-vector inequalities and inverse bounds.

Notes Good foundational perturbation task.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No `LeanFpAnalysis` imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

-- TODO: paste model output from natural-language Condition A run

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

-- TODO: paste model output from natural-language Condition C run

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions

Minimal statement context

-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof lemmas

Condition A proof

-- TODO: generated proof from fixed Lean theorem with minimal statement scaffold

Condition C library context

-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C

Condition C proof

-- TODO: generated proof from the exact same fixed Lean theorem with library context

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.18 Ch07 Ex. 7.7 – Backward Error Comparison

Tier Good

Theme Backward error comparison

Exercise prompt For the normwise and componentwise backward-error measures from the chapter, prove the comparison inequalities

$$\omega_{|A|,|b|}(y) \leq \omega_{|A|,0}(y) \leq \frac{2\omega_{|A|,|b|}(y)}{1 - \omega_{|A|,|b|}(y)}$$

and

$$\eta_{A,b}(y) \leq \eta_{A,0}(y) \leq \frac{2\eta_{A,b}(y)}{1 - \eta_{A,b}(y)}.$$

Benchmark intent Derive the displayed comparison inequalities.

Expected dependencies

Backward-error definitions and algebraic manipulation of bounds.

Notes

Useful for library-level error-measure definitions.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.19 Ch08 Ex. 8.3 – Triangular Solve Component Bound

Tier Best

Theme Triangular solve forward error

Exercise prompt Let $U \in \mathbb{R}^{n \times n}$ be unit upper triangular with $|u_{ij}| \leq 1$ for $j > i$. Using the triangular-solve theorem, show that the computed substitution solution $\hat{x}$ to $Ux = b$ satisfies

$$|x_i - \hat{x}_i| \leq 2^{n-i}((n^2 + n + 1)u + O(u^2))\|b\|_\infty.$$

Compare this with the result obtained from the more general triangular-solve bound.

Benchmark intent Prove the componentwise bound for unit upper triangular systems with bounded off-diagonals.

Expected dependencies

Triangular solve backward/forward bounds.

Notes Strong direct stability task.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.20 Ch08 Ex. 8.4 – Triangular M-Matrix Conditioning

Tier	Best
Theme	Componentwise conditioning
Exercise prompt	Let $T \in \mathbb{R}^{n \times n}$ be triangular, assume $T = M(T)$, and suppose $Tx = b \geq 0$. Prove $ T^{-1} T x \leq (2n - 1) x ,$ and hence $\text{cond}(T, x) \leq 2n - 1$.
Benchmark intent	Formalize the bound $\text{cond}(T, x) \leq 2n - 1$ under the stated hypotheses.
Expected dependencies	Triangular comparison and M-matrix inverse bounds.
Notes	Strong if the M-matrix definitions are stable.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.21 Ch08 Ex. 8.7 – Diagonal Dominance Inverse Bound

Tier Maybe

Theme Diagonal dominance bound

Exercise prompt For a strictly row diagonally dominant matrix satisfying

$$\alpha_i = |a_{ii}| - \sum_{j \neq i} |a_{ij}| > 0,$$

prove

$$\|A^{-1}\|_{\infty} \leq \frac{1}{\min_i \alpha_i}.$$

Optionally extend this to the scaled case AD with positive diagonal $D = \text{diag}(d_i)$, giving $\|A^{-1}\|_{\infty} \leq \|D\|_{\infty} / \min_i \beta_i$.

Benchmark intent Use only a scoped subpart, preferably the inverse-norm bound.

Expected dependencies

Matrix norm and diagonal dominance lemmas.

Notes More exact matrix analysis than floating-point stability.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No `LeanFpAnalysis` imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

-- TODO: paste model output from natural-language Condition A run

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

-- TODO: paste model output from natural-language Condition C run

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions

Minimal statement context

-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof lemmas

Condition A proof

-- TODO: generated proof from fixed Lean theorem with minimal statement scaffold

Condition C library context

-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C

Condition C proof

-- TODO: generated proof from the exact same fixed Lean theorem with library context

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.22 Ch09 Ex. 9.6 – Totally Nonnegative Growth Factor

Tier Good

Theme Growth factor

Exercise prompt If $A \in \mathbb{R}^{n \times n}$ is nonsingular and totally nonnegative, show that it has an LU factorization $A = LU$ with $L \geq 0$ and $U \geq 0$. Using the determinant inequality for totally nonnegative matrices, deduce that Gaussian elimination without pivoting has growth factor $\rho_n = 1$.

Benchmark intent Formalize the growth-factor consequence.

Expected dependencies

LU factorization; total nonnegativity; determinant inequalities.

Notes Specialized but relevant to GE stability.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.23 Ch09 Ex. 9.8 – Absolute LU Factorization

Tier Good

Theme LU backward structure

Exercise prompt If $A \in \mathbb{R}^{n \times n}$ is nonsingular and A^{-1} is totally nonnegative, show that A has an LU factorization $A = LU$ satisfying

$$|A| = |L||U|.$$

Benchmark intent Derive the LU absolute-value relation and stability implication.

Expected dependencies

LU exact algebra and sign-structure lemmas.

Notes Good support for growth-factor and stability reasoning.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.24 Ch09 Ex. 9.9 – GE Growth Bound Without Pivoting

Tier Good

Theme Growth factor bound

Exercise prompt For Gaussian elimination without pivoting, prove the growth-factor estimate

$$\rho_n \leq 1 + \frac{n \|L\| \|U\|_\infty}{\|A\|_\infty}.$$

Benchmark intent Prove the growth-factor inequality.

Expected dependencies

GE/LU definitions and max-norm inequalities.

Notes Good compact task if growth factor is already defined.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.25 Ch09 Ex. 9.13 – Threshold Pivoting Growth Bound

Tier Good

Theme Threshold pivoting

Exercise prompt In sparse Gaussian elimination with threshold pivoting, a pivot in column k is chosen so that

$$|a_{ik}^{(k)}| \geq \tau \max_{m \geq k} |a_{mk}^{(k)}|, \quad \tau \in (0, 1].$$

Show that

$$\max_i |a_{ij}^{(k)}| \leq (1 + \tau^{-1})^{\mu_j} \max_i |a_{ij}|,$$

where μ_j is the number of nonzeros in the j th column of U , and deduce a growth-factor bound.

Benchmark intent Prove the per-column growth recurrence and resulting growth-factor bound.

Expected dependencies

Sparse column counts; threshold pivoting definitions; growth-factor algebra.

Notes Good later benchmark; definitions may need local setup.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No `LeanFpAnalysis` imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

-- TODO: paste model output from natural-language Condition A run

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

-- TODO: paste model output from natural-language Condition C run

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions

Minimal statement context

-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof lemmas

Condition A proof

-- TODO: generated proof from fixed Lean theorem with minimal statement scaffold

Condition C library context

-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C

Condition C proof

-- TODO: generated proof from the exact same fixed Lean theorem with library context

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.26 Ch09 Ex. 9.14 – Pre-Pivoted GEPP Equivalence

Tier Maybe

Theme Pivoting equivalence

Exercise prompt Let A be pre-pivoted for GEPP, meaning GEPP performs no row interchanges on A . Show GEPP computes the same LU factorization as the first pivoting method from the chapter and as pairwise pivoting applied to ΠA , where Π is the row-reversal permutation, using the natural strategy that keeps multipliers bounded by 1.

Benchmark intent Use only if scoped to a stability consequence.

Expected dependencies

Permutation matrices and GEPP definitions.

Notes Mostly structural, so lower priority.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.27 Ch19 Ex. 19.2 – Computed Householder Orthogonality

Tier Best

Theme Orthogonality error

Exercise prompt Let $\hat{P} = I - \hat{\beta} \hat{v} \hat{v}^T$, where $\hat{\beta}$ and $\hat{v}$ are the computed Householder quantities from Lemma 19.1. Derive a bound for

$$\|\hat{P}^T \hat{P} - I\|_2.$$

Benchmark intent Prove a bound for $\hat{P}^T \hat{P} - I$ using the computed beta/vector error model.

Expected dependencies

Householder reflector definitions and norm inequalities.

Notes Very good QR benchmark.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No `LeanFpAnalysis` imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.28 Ch19 Ex. 19.6 – Householder QR One-Step Error

Tier	Best
Theme	Householder update error
Exercise prompt	In Householder QR, prove the vector-size bound for the kth Householder vector, $\sqrt{2} \ a_k^{(k)}(k : m)\ _2 \leq \ v_k\ _2 \leq 2 \ a_k^{(k)}(k : m)\ _2.$ Then, for $j > k$, analyze the computed update $\hat{a}_j^{(k+1)} = \text{fl}(\hat{P}_k \hat{a}_j^{(k)})$, where $\hat{P}_k = I - \beta_k \hat{v}_k \hat{v}_k^T$ and $\Delta v_k \leq \tilde{\gamma}_{m-k} v_k$. Show $\hat{a}_j^{(k+1)} = P_k \hat{a}_j^{(k)} + f_j^{(k)},$ with the leading zero pattern in $f_j^{(k)}$ and the displayed componentwise bound involving $u \hat{a}_j^{(k)}$ and $\tilde{\gamma}_{m-k} v_k$.
Benchmark intent	Prove the residual form and componentwise/normwise bound for a panel update.
Expected dependencies	Householder vector bounds; reflector application error.
Notes	Excellent because it composes several QR facts.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.29 Ch19 Ex. 19.10 – CGS/MGS Orthogonality Example

Tier	Good
Theme	Gram-Schmidt instability
Exercise prompt	For the Lauchli-type matrix

$$A = \begin{bmatrix} 1 & 1 & 1 \\ \epsilon & 0 & 0 \\ 0 & \epsilon & 0 \\ 0 & 0 & \epsilon \end{bmatrix},$$

assuming $\text{fl}(1 + \epsilon^2) = 1$, evaluate the Q matrices produced by classical Gram-Schmidt and modified Gram-Schmidt, and assess their orthonormality.

Benchmark intent	Formalize the computed Q matrices under a rounding assumption and bound orthogonality loss.
-------------------------	---

Expected dependencies	Gram-Schmidt algorithm definitions.
------------------------------	-------------------------------------

Notes	Good as an example task, but less general than 19.6.
--------------	--

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.30 Ch19 Ex. 19.12 – QR Backward-Error Structure

Tier	Good
Theme	Backward-error structure
Exercise prompt	Assume matrices satisfy

$$\begin{bmatrix} A + \Delta A_1 \\ \Delta A_2 \end{bmatrix} = \begin{bmatrix} P_{11} \\ P_{21} \end{bmatrix} R, \quad P_{11}^T P_{11} + P_{21}^T P_{21} = I,$$

with P_{11}, P_{21} having at least as many rows as columns. Show that there exists an orthonormal Q such that

$$A + \Delta A = QR, \quad \Delta A = F \Delta A_1 + \Delta A_2, \quad \|F\|_2 \leq 1.$$

Benchmark intent Prove existence of an orthonormal factor and a combined data perturbation.

Expected dependencies

CS decomposition or a scoped substitute; polar/orthogonal factor facts.

Notes Powerful but may require heavier exact linear algebra.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement


```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.31 Ch20 Ex. 20.2 – Householder QR Least-Squares Backward Stability

Tier Best

Theme Least-squares backward stability

Exercise prompt Prove the Householder QR least-squares stability theorem: if $A \in \mathbb{R}^{m \times n}$, $m \geq n$, has full rank and $\min_x \|b - Ax\|_2$ is solved by Householder QR, then the computed $\hat{x}$ is the exact least-squares solution to

$$\min_x \|(b + \Delta b) - (A + \Delta A)x\|_2,$$

with columnwise and right-hand-side bounds

$$\|\Delta a_j\|_2 \leq \tilde{\gamma}_{mn} \|a_j\|_2, \quad \|\Delta b\|_2 \leq \tilde{\gamma}_{mn} \|b\|_2.$$

Benchmark intent Show the computed LS solution exactly solves a perturbed LS problem with columnwise/RHS perturbation bounds.

Expected dependencies

Householder QR backward error; triangular solve backward error; LS normal-equation equivalence.

Notes One of the best benchmark tasks.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.32 Ch20 Ex. 20.5 – MGS Least-Squares Backward Stability

Tier	Best
Theme	MGS LS backward stability
Exercise prompt	For the modified Gram-Schmidt least-squares method applied as described in the chapter, prove an analogue of the Householder QR theorem: the computed $\hat{x}$ is the exact solution of a nearby least-squares problem, with normwise or columnwise perturbation bounds of the same qualitative form as in Theorem 20.3.
Benchmark intent	Formalize the MGS LS backward-stability result under a scoped MGS model.
Expected dependencies	MGS QR model; LS backward-error definitions.
Notes	Good if MGS algorithm support exists or is supplied in the task.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.33 Ch20 Ex. 20.6 – Normal Equations Residual Error

Tier Best

Theme Residual error

Exercise prompt For $\min_x \|b - Ax\|_2$, let $\hat{x}$ be the solution computed by the normal-equations method, let x be the exact solution, and define $\hat{r} = b - A\hat{x}$, $r = b - Ax$. Using the preceding normal-equations error bounds, prove a first-order estimate of the form

$$|r^T r - \hat{r}^T \hat{r}| \leq c_{m,n} u \|x - \hat{x}\|_2 \|A\|_2 (\|A\|_2 \|\hat{x}\|_2 + \|b\|_2) + O(u^2).$$

Benchmark intent Derive a bound for $|r^T r - \hat{r}^T \hat{r}|$.

Expected dependencies

Normal-equations backward error and residual algebra.

Notes Strong because it is not just a direct theorem restatement.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

-- TODO: paste model output from natural-language Condition A run

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

-- TODO: paste model output from natural-language Condition C run

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions

Minimal statement context

-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof lemmas

Condition A proof

-- TODO: generated proof from fixed Lean theorem with minimal statement scaffold

Condition C library context

-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C

Condition C proof

-- TODO: generated proof from the exact same fixed Lean theorem with library context

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.34 Ch20 Ex. 20.8 – LS Backward Error with Fixed Right-Hand Side

Tier	Good
Theme	LS backward error
Exercise prompt	For the normwise least-squares backward error $\eta_F(y)$ of Theorem 20.5, evaluate the limiting case $\theta = \infty$, i.e. the case $\Delta b \equiv 0$ where only A may be perturbed.
Benchmark intent	Derive the specialized expression or bound.
Expected dependencies	LS backward-error formula and singular-value/eigenvalue facts.
Notes	Good if the full formula has already been formalized.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.35 Ch20 Ex. 20.9 – Stable LS Backward-Error Formula

Tier Good

Theme Stable formula

Exercise prompt Prove the stable formula for the least-squares backward error:

$$\eta_F(y) = \min \{ \phi, \sigma_{\min}([A \ \phi(I - rr^+)]) \}, \quad \phi = \sqrt{\mu} \frac{\|r\|_2}{\|y\|_2},$$

where $r = b - Ay$, $\mu = \theta^2 \|y\|_2^2 / (1 + \theta^2 \|y\|_2^2)$, and r^+ denotes the pseudoinverse of r viewed as a vector.

Benchmark intent Prove equivalence of the alternative formula to the original backward-error expression.

Expected dependencies

Singular-value formulas and projection residual algebra.

Notes Good but may be exact-linear-algebra heavy.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No `LeanFpAnalysis` imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

-- TODO: paste model output from natural-language Condition A run

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

-- TODO: paste model output from natural-language Condition C run

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

-- TODO: theorem or lemma statement used verbatim in both fixed-theorem conditions

Minimal statement context

-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof lemmas

Condition A proof

-- TODO: generated proof from fixed Lean theorem with minimal statement scaffold

Condition C library context

-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C

Condition C proof

-- TODO: generated proof from the exact same fixed Lean theorem with library context

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.36 Ch20 Ex. 20.11 – LSE First-Order Reduction

Tier Good

Theme First-order approximation

Exercise prompt For the equality-constrained least-squares perturbation bound, specialize the general formula to the unconstrained problem $B = 0$, $d = 0$. Show that the bound reduces, to first order, to the standard unconstrained least-squares condition estimate.

Benchmark intent Formalize the reduction under the unconstrained specialization.

Expected dependencies

First-order perturbation framework and LS definitions.

Notes Useful if LSE definitions are available.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.37 Ch22 Ex. 22.8 – Bidiagonal Inverse Perturbation

Tier Best

Theme Bidiagonal inverse perturbation

Exercise prompt Let $U \in \mathbb{R}^{n \times n}$ be upper bidiagonal and let $\Delta u_{ij} = \delta_{ij} u_{ij}$. Derive the entry-wise formula for $(U + \Delta U)^{-1} - U^{-1}$ in terms of the diagonal and superdiagonal δ 's. Then apply it to the perturbed bidiagonal factors $U_k + \Delta U_k$ from the Vandermonde analysis to prove a bound of the form

$$(U_k + \Delta U_k)^{-1} = U_k^{-1} + F_k, \quad |F_k| \leq \frac{(n+1)u}{1 - (n+1)u} |U_k^{-1}|.$$

Benchmark intent Prove the exact formula and deduce the displayed bound.

Expected dependencies

Bidiagonal inverse formulas; product perturbation bounds.

Notes Excellent structured-matrix stability task.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No `LeanFpAnalysis` imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.38 Ch22 Ex. 22.11 – Structured Vandermonde Condition Number

Tier Best

Theme Structured condition number

Exercise prompt For the primal Vandermonde system $Vx = b$, where $V = V(\alpha_0, \dots, \alpha_n)$, define a structured condition number by allowing relative perturbations $|\Delta\alpha_i| \leq \epsilon|\alpha_i|$. Show

$$\text{cond}(V) = \frac{\|V^{-1} \text{diag}(0, 1, \dots, n)V\|_x\|_\infty}{\|x\|_\infty},$$

and derive the corresponding condition number for the dual system $V^T a = f$.

Benchmark intent Formalize the first-order condition number formula for the primal system and optionally the dual analogue.

Expected dependencies

Vandermonde matrices; derivative of powers; componentwise perturbation.

Notes Strong, but likely needs local Vandermonde definitions.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.39 Ch22 Ex. 22.3 – Vandermonde Inverse Algorithm Stability

Tier Good

Theme Algorithm stability

Exercise prompt For the Vandermonde inverse algorithm and its modified version, evaluate left and right residuals of the computed inverses and compare them with GEPP. As a scoped formal task, prove the deterministic sign/cancellation claim: the algorithm performs subtractions of like-signed numbers, and use that fact to state the resulting stability implication.

Benchmark intent Use a scoped deterministic residual or subtraction-sign result, not the research subpart.

Expected dependencies

Vandermonde inverse algorithm definitions.

Notes Avoid empirical or open-ended wording.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.40 Ch22 Ex. 22.7 – Chebyshev-Vandermonde Conditioning

Tier Good

Theme Conditioning

Exercise prompt For the Chebyshev-Vandermonde-like matrix $T = T(\alpha_0, \dots, \alpha_n)$, prove the condition estimates

$$\kappa_2(T) = \sqrt{2} \quad \text{for} \quad \alpha_i = \cos((i + \tfrac{1}{2})\pi/(n + 1)),$$

and

$$\kappa_2(T) \leq 2 \quad \text{for} \quad \alpha_i = \cos(i\pi/n).$$

Benchmark intent Prove one of the two condition-number bounds.

Expected dependencies

Chebyshev polynomial identities and discrete orthogonality.

Notes More exact approximation theory than FP stability, but condition-focused.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.41 Ch23 Ex. 23.6 – 3M plus Strassen Error Bound

Tier Best

Theme Fast matrix multiply error

Exercise prompt Combine the complex $3M$ multiplication formula with Strassen multiplication for the real products. Prove the normwise error bound described after Section 23.2.4: the computed complex product satisfies the Strassen bound analogous to

$$\|C - \hat{C}\| \leq \left[\left(\frac{n}{n_0} \right)^{\log_2 12} (n_0^2 + 5n_0) - 5n \right] u \|A\| \|B\| + O(u^2),$$

but with the extra constant multiplier and additive constant introduced by the $3M$ combination.

Benchmark intent Derive the Strassen-style bound with the extra constants from the $3M$ combination.

Expected dependencies

Strassen normwise bound; $3M$ component formulas; complex matrix norms.

Notes Excellent for showing normwise versus componentwise stability.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.42 Ch24 Ex. 24.1 – FFT Convolution Integer Recovery

Tier Best

Theme FFT convolution error

Exercise prompt For integer vectors x, y , form the convolution coefficients z_k by padding with zeros and using

$$z = F_n^{-1}((F_n x) \cdot (F_n y)),$$

where the dot denotes componentwise multiplication. Derive a bound on $z - \hat{z}$ for the FFT-based computation and give a sufficient condition under which componentwise nearest-integer rounding of $\hat{z}$ recovers the exact integer vector z .

Benchmark intent Prove that nearest-integer recovery is exact under a sufficient error bound.

Expected dependencies

DFT matrix norm/error model; componentwise multiplication error; inverse transform bound.

Notes Strong but may require FFT/DFT local setup.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.43 Ch25 Ex. 25.1 – Perturbed Contraction Iteration

Tier Good

Theme Perturbed iteration stability

Exercise prompt Consider

$$x_{k+1} = G(x_k) + e_k, \quad \|e_k\| \leq \alpha,$$

where $G : \mathbb{R}^n \rightarrow \mathbb{R}^n$ satisfies

$$\|G(x) - a\| \leq \beta \|x - a\|, \quad 0 < \beta < 1.$$

Show the ball of radius $\alpha/(1 - \beta)$ about a is invariant, that outside this ball the distance to a strictly decreases, and that every accumulation point lies in the ball.

Benchmark intent Prove the invariant ball and accumulation-point bound.

Expected dependencies

Norm contraction algebra and recurrence bounds.

Notes

Good nonlinear stability task, less FP-specific.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.44 Ch27 Ex. 27.5 – Robust Scaled 2-Norm

Tier Good

Theme Robust norm computation

Exercise prompt Analyze the scaled BLAS $xNRM2$ algorithm for $\|x\|_2$. It keeps t as the current scale and s as the scaled sum of squares, updating by

$$\text{if } |x_i| > t : \quad s := 1 + s(t/x_i)^2, \quad t := |x_i|, \quad \text{else } s := s + (x_i/t)^2,$$

and returns $t\sqrt{s}$. Prove the invariant establishing correctness and state the robustness properties regarding overflow and underflow.

Benchmark intent Formalize the invariant relating the scale and accumulated sum to $\|x\|_2$.

Expected dependencies

Norm-square algebra; overflow/underflow-free exact model if included.

Notes Good algorithmic task, but not central matrix stability.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.

6.45 Ch27 Ex. 27.6 – Pythagorean Sum Iteration

Tier Good

Theme Robust Pythagorean sum

Exercise prompt For $0 < y_0 \leq x_0$, let $p = \sqrt{x_0^2 + y_0^2}$. Show that Halley’s method applied to $f(x) = x^2 - p^2$ gives

$$x_{n+1} = x_n \left(1 + 2 \frac{p^2 - x_n^2}{p^2 + 3x_n^2} \right),$$

that $x_n \leq x_{n+1} \leq p$, and, with $y_n = \sqrt{p^2 - x_n^2}$, derive the scaled recurrence implemented by the `pythag` algorithm. Also prove the cubic error identity

$$p - x_{n+1} = \frac{(p - x_n)^3}{p^2 + 3x_n^2}.$$

Benchmark intent Prove monotone/cubic convergence and the scaled recurrence.

Expected dependencies

Scalar iteration algebra and `norm2/sqrt` support.

Notes Good robust-arithmetic task; keep the floating-point robustness claim scoped.

Layer 1: natural-language discovery run

This run measures whether the model can turn the exercise prompt into a useful Lean theorem and proof.

Condition A setup

Exercise prompt plus only the neutral Lean scaffold needed to state the task: basic types, notation, and algorithm definitions if required. No LeanFpAnalysis imports, source ledger, stability bounds, proof lemmas, or proof logic.

Condition A generated theorem and proof

```
-- TODO: paste model output from natural-language Condition A run
```

Condition C setup

Same exercise prompt, but with LeanFpAnalysis imports, source-oriented library documentation, and available project context.

Condition C generated theorem and proof

```
-- TODO: paste model output from natural-language Condition C run
```

Discovery assessment

TODO: record whether the two generated Lean statements prove the same mathematical theorem, whether either statement is too weak or has hidden assumptions, and whether the result is eligible for the fixed-theorem run.

Layer 2: fixed Lean theorem run

This run measures proof assistance only. The canonical theorem below must be frozen before either condition is run.

Canonical Lean theorem statement

```
-- TODO: theorem or lemma statement used verbatim in both fixed-theorem  
conditions
```

Minimal statement context

```
-- TODO: neutral stubs/imports needed to state the theorem; no bounds or proof  
lemmas
```

Condition A proof

```
-- TODO: generated proof from fixed Lean theorem with minimal statement  
scaffold
```

Condition C library context

```
-- TODO: LeanFpAnalysis imports/docs/lemmas made visible for Condition C
```

Condition C proof

```
-- TODO: generated proof from the exact same fixed Lean theorem with library  
context
```

Fixed-theorem assessment

TODO: record success or failure, proof length, wall time, use of source-matched library lemmas, hidden assumptions, remaining `sorry`s, and whether the proof is implementation-backed or contract-transfer.

Natural-language explanation of accepted proof

TODO.